

DEVOPS CODE SMELL VISUALIZATION DASHBOARD

Project Report

Submitted by: Jayapramod Manikantan

Program: B.Tech Computer Science and Engineering

Specialization: DevOps

A simple automated platform for analyzing source-code quality, detecting code smells, visualizing technical debt, and integrating code-quality checks into CI/CD.

1. Introduction

The DevOps Code Smell Visualization Dashboard is a software-quality monitoring platform designed to analyze source-code repositories and present code-quality information through an interactive dashboard. The system combines static code analysis, backend APIs, database storage, visualization, and CI/CD automation.

The main objective is to make code smells and maintainability issues easier to identify and track. Instead of manually inspecting large repositories, developers can run an automated analysis and view the results in a centralized dashboard.

2. Problem Statement

Large software repositories can contain long functions, complex code, large classes, duplicated code, excessive parameters, and other maintainability issues. These problems can become harder to identify as a project grows. Traditional code inspection is time-consuming and may not provide a clear historical view of code quality.

This project addresses the problem by automatically analyzing repositories, identifying measurable code smells, storing analysis results, and presenting them using charts, tables, filters, and historical trends.

3. Objectives

- Automatically analyze source-code repositories.
- Detect common code smells using static analysis techniques.
- Calculate useful source-code metrics such as lines of code and complexity.
- Store analysis results for later comparison.
- Provide an interactive visualization dashboard.
- Track code-quality trends across analysis runs or commits.
- Integrate code-quality checks with a Jenkins CI/CD pipeline.
- Use Docker and Kubernetes for deployment and scalability.

4. Proposed System

The proposed system follows an automated pipeline. A repository is selected for analysis, the source code is processed by an analyzer, detected smells and metrics are converted into structured data, and the results are stored in a database. A backend API provides the data to the frontend dashboard.

High-Level Architecture

Component	Purpose
GitHub / Git repository	Source-code repository to be analyzed
Jenkins	Automates analysis, testing, quality gates, and build steps
Analyzer	Parses source code and detects code smells and metrics
PostgreSQL	Stores repositories, analysis runs, files, smells, and metrics
FastAPI	Provides REST APIs for analysis data
React Dashboard	Displays charts, tables, filters, and code-quality trends
Docker	Packages application components into containers
Kubernetes	Deploys and manages application containers

Component	Purpose
Prometheus / Grafana	Optional monitoring and operational observability

5. Code Smell Detection

The analyzer can use Python AST-based analysis together with established static-analysis tools. The Abstract Syntax Tree (AST) represents source code as a structured tree, allowing the analyzer to inspect functions, classes, parameters, branches, loops, and other program structures.

Initial Code Smells

Code smell	Example detection metric
Long Function	Function length exceeds configured threshold
Large Class	Class contains excessive lines or methods
Too Many Parameters	Function exceeds parameter threshold
High Complexity	Cyclomatic complexity exceeds threshold
Deep Nesting	Nested control structures exceed threshold
Long File	File contains excessive lines
Duplicate Code	Similar code blocks exceed similarity threshold
TODO / FIXME	Unresolved maintenance comments

6. Data Model

The database is designed around repositories and analysis runs so that code quality can be compared over time.

Table	Important fields
repositories	id, name, URL, branch, commit hash, analyzed time
files	id, repository ID, path, language, lines
smells	id, repository ID, file ID, smell type, severity, line, metric, value
analysis_runs	id, repository ID, commit hash, start time, end time, total files, total smells

7. Dashboard Features

The dashboard is planned with the following major views:

- Overview: total files, lines of code, code smells, severity distribution, and overall metrics.
- Code Smells: searchable and filterable list of detected smells with file and line information.
- Files: file-level metrics including lines of code, complexity, and smell count.
- Complexity: distribution of complexity values and functions with higher complexity.
- Trends: historical changes in smell counts and other quality metrics across analysis runs.
- Source Details: detailed information about the affected file, function/class, metric, threshold, and source-code location.

8. CI/CD Integration

Jenkins is used to automate the analysis as part of the software delivery process. A typical pipeline performs source checkout, dependency installation, tests, static analysis, code-smell analysis, quality-gate evaluation, Docker image creation, and deployment.

Proposed Pipeline

Stage	Activity
1. Checkout	Clone the repository and select the required branch
2. Test	Run automated tests
3. Code Quality	Run static-analysis tools
4. Smell Analysis	Run the custom code-smell analyzer
5. Quality Gate	Check configured quality thresholds
6. Docker Build	Build application container images
7. Push	Push images to a container registry
8. Deploy	Deploy the application to the target environment

9. Quality Gate

A quality gate prevents a build from progressing when important quality requirements are not satisfied. Example thresholds can include zero critical issues, a maximum number of high-severity smells, acceptable average complexity, and minimum test coverage. These thresholds should be configurable rather than hard-coded.

10. Containerization and Kubernetes

Docker can package the frontend, backend, analyzer, and supporting services into reproducible containers. Kubernetes can then manage deployments, services, secrets, ingress, persistent storage, and horizontal scaling. This allows the project to demonstrate practical DevOps deployment concepts.

11. Monitoring

Prometheus and Grafana can optionally be added to monitor API requests, response time, analyzer execution time, failed analyses, container CPU and memory usage, and other operational metrics. This extends the project from code-quality analysis into application observability.

12. Expected Outcome

The completed system will provide an automated way to inspect source-code quality and visualize technical debt. Developers will be able to identify where code smells occur, understand their severity and associated metrics, compare repositories or analysis runs, and observe quality trends over time.

13. Future Enhancements

- Support for additional programming languages such as JavaScript, Java, and TypeScript.
- GitHub webhook-based automatic analysis after every push or pull request.
- Pull-request quality reports and comments.
- More advanced duplicate-code and dependency analysis.
- Cloud deployment using AWS services such as EKS, RDS, and a container registry.

- Role-based access control and repository-level permissions.

14. Conclusion

The DevOps Code Smell Visualization Dashboard combines software-quality analysis with modern DevOps practices. By connecting automated static analysis to a database, REST API, visualization layer, CI/CD pipeline, containerization, and Kubernetes deployment, the project demonstrates how code quality can become a measurable and continuously monitored part of the software delivery lifecycle.